

Evaluación 2 - Proyecto

Grafo por nodos y enlaces + coloración (método greedy)
Leonardo Balmes Solis, Isis y Scarlet

Objetivo del Proyecto

- Implementar un grafo no dirigido.
- Representarlo con lista de adyacencia.
- Aplicar el algoritmo greedy para colorear vértices.
- Validar automáticamente la solución.
- Comparar distintos órdenes de coloreo.

¿Qué es un Grafo?

Un grafo está compuesto por:

- Vértices (nodos)
- Aristas (conexiones)

En este proyecto se implementó un **grafo no dirigido**, lo que significa:

Si A está conectado con B, entonces B también está conectado con A.

```
{  
  "A": ["B", "C"],  
  "B": ["A"],  
  "C": ["A"]  
}
```

Algoritmo Greedy Coloring

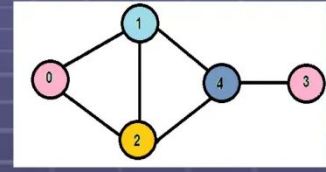
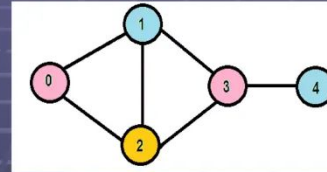
¿Cómo funciona?

1. Se toma un vértice.
2. Se revisan los colores de sus vecinos.
3. Se asigna el color más pequeño disponible.
4. Se repite hasta colorear todos.

No permite que vecinos tengan el mismo color.

No siempre garantiza el mínimo número posible de colores.

Analysis of Basic Algorithm



Prueba 1: Grafo Completo K4

Características:

- 4 vértices
- Todos conectados entre sí

Resultado:

- Total de colores usados: 4
- Validación: Válida

```
▶ # 1. Instanciamos un objeto de la clase Graph
g1 = Graph()

# 2. Definimos la lista de vértices para el grafo completo
vertices = ["A", "B", "C", "D"]

# 3. Construcción del Grafo Completo K4
# Utilizamos un ciclo anidado para conectar cada nodo con todos los que le siguen en la lista
for i in range(len(vertices)):
    for j in range(i + 1, len(vertices)):
        # Agrega una arista entre cada par de nodos único
        g1.add_edge(vertices[i], vertices[j])

# 4. Aplicamos el algoritmo de coloración voraz (Greedy)
coloring_k4 = g1.greedy_coloring()

# 5. Despliegue de resultados en consola
print("K4")
for v in coloring_k4:
    # Muestra la asignación: Vértice > Color (0, 1, 2 o 3)
    print(v, ">", coloring_k4[v])

# 6. Cálculo del total de colores únicos utilizando un conjunto (set)
print("Total colores usados:", len(set(coloring_k4.values())))
```

Prueba 2: Ciclo C5

Características:

- 5 vértices en forma circular

Resultado:

- Total de colores usados: 3
- Validación: Válida

```
# 1. Instanciamos un segundo objeto de la clase Graph para la prueba del ciclo
g2 = Graph()

# 2. Definimos los vértices que formarán el anillo
cycle = ["A", "B", "C", "D", "E"]

# 3. Construcción del Grafo Ciclo C5
# El uso del operador módulo (%) es clave aquí: conecta cada nodo con el siguiente,
# y al llegar al último (E), lo conecta de regreso al primero (A) para cerrar el círculo.
for i in range(len(cycle)):
    g2.add_edge(cycle[i], cycle[(i + 1) % 5])

# 4. Ejecutamos el coloreo voraz sobre la estructura circular
coloring_c5 = g2.greedy_coloring()

# 5. Despliegue de la asignación de colores
print("C5")
for v in coloring_c5:
    print(v, ">", coloring_c5[v])

# 6. Verificamos la eficiencia: en un C5, el resultado esperado es 3 colores
print("Total colores usados:", len(set(coloring_c5.values())))

# 7. Comprobación de que la solución respeta las reglas del grafo
print("Validación:", g2.validate_coloring(coloring_c5))
```

```
... C5
A > 0
B > 1
C > 0
D > 1
E > 2
Total colores usados: 3
Validación: Válida
```

Conclusión

El algoritmo greedy es eficiente

Depende del orden de procesamiento

Es sencillo de implementar

Funciona correctamente para los grafos probados